

LAYERZERO - SECURITY REVIEW

LayerZero Solana Console.

A security review of LayerZero's Solana Console.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
LayerZero	Solana Console	Osman Ozdemir, Ali Shehab, Zdravko Hristov, Minato7namikazi	May 11 - July 15, 2026	July 15, 2026

§ 00

Contents.

55 pages - overview, confidence ranking, scope, 42 findings, disclaimer, and about Guardian.

§ 01	Overview	p. 03
§ 02	Confidence Ranking	p. 04
§ 03	Scope & Classifications	p. 05
	Files in Scope	p. 06
§ 04	Findings	p. 07
	Round 1 - Main Review · 15 findings	p. 11-25
	Round 2 - Remediation Review · 23 findings	p. 26-49
	Round 3 - Remediation Review · 4 findings	p. 50-53
§ 05	Disclaimer	p. 54
§ 06	About Guardian	p. 55

Overview.

Guardian reviewed LayerZero Solana Console across the main review and two remediation rounds. The final report includes 6 Low findings, 36 Info findings.

AUDIT FIRM	Guardian	REPOSITORY	LayerZero-Labs/audit-external
CLIENT	LayerZero	METHODOLOGY	Smart Contract - Manual - Remediation Review
PROJECT	Solana Console	MAIN REVIEW COMMIT	bf9ea5e4db2e56f8be4ea2718d84b926d14d8729
PREPARED BY	Osman Ozdemir, Ali Shehab, Zdravko Hristov, Minato7namikazi	REMEDIATION COMMIT	5cc7cacbaad6902eb00e6741f1514354747def15
AUDIT WINDOW	May 11 - July 15, 2026		
FINAL REPORT	July 15, 2026		

1.1 Confidence Ranking

4/5

High Confidence

Guardian assigns a Confidence Ranking of 4 out of 5. The score reflects the provided final confidence rating, the predominantly informational finding profile, and the resolved findings from the final remediation round.

1 V. LOW	2 LOW	3 MED	4 HIGH	5 V. HIGH
-------------	----------	----------	-----------	--------------

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	IN PROG	DECLINED	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0	0	0
High	0	0	0	0	0	0	0
Medium	0	0	0	0	0	0	0
Low	6	0	0	0	1	0	5
Info	36	0	0	0	3	0	33

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: Code is highly secure at time of audit. Low risk of latent critical issues.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence ASSIGNED	Code is clean, well-structured, and adheres to best practices. Only 1 Significant issue was uncovered per week. Design patterns are sound, and test coverage is strong. Recommendation: Suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity issues. Varied Low severity findings.
3 Moderate Confidence	Medium-severity and occasional High-severity issues found. Code is functional, but there are concerning areas such as weak modularity or risky patterns. No critical design flaws were identified, though some patterns could lead to edge-case issues. Recommendation: Address issues thoroughly and consider a targeted follow-up audit depending on code changes.	1-2 High/Critical findings per engagement week.
2 Low Confidence	Code shows frequent emergence of Critical/High vulnerabilities. Audit revealed recurring anti-patterns, weak test coverage, or unclear logic. These characteristics suggest a high likelihood of latent issues. Recommendation: Post-audit development and a second audit cycle are strongly advised.	2-4 High/Critical findings per engagement week, or additional unresolved High/Critical findings uncovered in remediation review.
1 Very Low Confidence	Code has systemic issues. Multiple High/Critical findings (≥ 5/week), poor security posture, and design flaws that introduce compounding risks. Safety cannot be assured. Recommendation: Halt deployment and seek a comprehensive re-audit after substantial refactoring.	≥ 5 High/Critical findings and overall systemic flaws.

§ 03

Scope & Classifications.

Guardian reviewed the LayerZero Solana Console code in `LayerZero-Labs/audit-external`, covering Solana OFT, Console OFT, transfer-hook, rate-limit, fee, pause, event, codec, and macro surfaces. The exported scope includes 1,483 source lines across 67 Rust/Solana files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MEDIUM	IMP · LOW	IMPACT · HIGH	Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.
LIK · HIGH	Critical	High	Medium	IMPACT · MEDIUM	Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.
LIK · MED	High	Medium	Low	IMPACT · LOW	Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.
LIK · LOW	Medium	Low	Low	LIKELIHOOD · HIGH	Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.
				LIKELIHOOD · MEDIUM	Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.
				LIKELIHOOD · LOW	Unlikely to ever occur in production.

Scope.

Files in scope, grouped by repository. SLOC counts are physical source lines.

LayerZero-Labs/audit-external (1/2)		34 FILES · 1159 SLOC	LayerZero-Labs/audit-external (2/2)		33 FILES · 324 SLOC
oft-ext/src			oft-ext/macros/ix/rate_limiter		
oft-extended_info.rs	205		get_rate_limit_global_config.rs	11	
oft/src			get_rate_limit_usages.rs	11	
oft_info.rs	168		is_rate_limit_address_exempt.rs	11	
oft-ext/src			rate_limits.rs	11	
types.rs	91		set_rate_limit_address_exemption.rs	11	
xfer-hook/src			set_rate_limit_config.rs	11	
lib.rs	77		set_rate_limit_global_config.rs	11	
oft-ext/src			set_rate_limit_state.rs	11	
events.rs	53		oft/macros/ix		
oft/src			quote_oft.rs	11	
types.rs	53		quote_send.rs	11	
xfer-hook/src			send.rs	11	
events.rs	50		shared_decimals.rs	11	
xfer-hook/macros/ix			token.rs	11	
mod.rs	37		xfer-hook/macros/ix/allowlist		
oft-ext/macros/ix			allowlist_mode.rs	11	
mod.rs	36		is_whitelisted.rs	11	
oft-ext/macros/src			set_allowlist_mode.rs	11	
oft-extended.rs	36		set_blacklisted.rs	11	
oft/macros/src			set_whitelisted.rs	11	
oft.rs	33		xfer-hook/macros/ix		
xfer-hook/src			get_version.rs	11	
params.rs	26		initialize.rs	11	
xfer-hook/macros/src			xfer-hook/macros/ix/pause		
transfer_hook.rs	24		is_paused.rs	11	
oft/src			pause_config.rs	11	
events.rs	23		set_default_paused.rs	11	
oft-ext/src			set_paused.rs	11	
lib.rs	22		xfer-hook/macros/ix/token		
oft/macros/ix			unregister_token_bypass.rs	11	
mod.rs	16		unregister_token_mint.rs	11	
xfer-hook/tests			oft-ext/macros/ix/rate_limiter		
allowlist_mode.rs	16		mod.rs	9	
oft-ext/macros/src			xfer-hook/macros/ix/allowlist		
lib.rs	13		mod.rs	7	
oft/macros/src			oft-ext/macros/ix/fee_config		
lib.rs	13		mod.rs	6	
xfer-hook/macros/src			oft/src		
lib.rs	13		lib.rs	5	
oft-ext/macros/ix/fee_config			xfer-hook/macros/ix/pause		
default_fee_bps.rs	11		mod.rs	5	
fee_bps.rs	11		xfer-hook/macros/ix/token		
get_amount_before_fee.rs	11		mod.rs	4	
get_fee.rs	11		oft-ext/macros/ix/fee_handler		
set_default_fee_bps.rs	11		mod.rs	2	
set_fee_bps.rs	11				
oft-ext/macros/ix/fee_handler					
fee_deposit.rs	11				
set_fee_deposit.rs	11				
oft-ext/macros/ix/pause					
default_paused.rs	11				
is_paused.rs	11				
pause_config.rs	11				
set_default_paused.rs	11				
set_paused.rs	11				
oft-ext/macros/ix/rate_limiter					
checkpoint_rate_limit.rs	11				

§ 04

Findings.

42 findings across three rounds: 6 Low findings, 36 Info findings. Findings are grouped by round, and visible IDs reset by severity within each round.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
ROUND 1 - MAIN REVIEW					
I-01	Allowlist Mode ABI Uses Raw u8	Compatibility	INFO	RESOLVED	p. 11
I-02	Parity doc misdescribes rate_limits return type	Documentation	INFO	RESOLVED	p. 12
I-03	FeeDepositSet doc names nonexistent EVM event	Documentation	INFO	RESOLVED	p. 13
I-04	Parity doc references stale scale_decimals field	Documentation	INFO	RESOLVED	p. 14
I-05	OftInfo.oft_type raw u8 bypasses enum guard	Best Practices	INFO	RESOLVED	p. 15
I-06	Undocumented RoleType requirement in #[oft]	Documentation	INFO	RESOLVED	p. 16
I-07	Rate limiter doc names nonexistent EVM file	Documentation	INFO	RESOLVED	p. 17
I-08	getRateLimitUsages doc misnames EVM params	Documentation	INFO	RESOLVED	p. 18
I-09	Unqualified Pubkey Makes Expansion Import-Sensitive	Compatibility	INFO	ACKNOWLEDGED	p. 19
I-10	Inconsistent OFT type docs	Documentation	INFO	RESOLVED	p. 20
I-11	Misleading oft_app comment	Documentation	INFO	RESOLVED	p. 21

§ 04

Findings, continued.

Index page 2 of 4.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
ROUND 1 - MAIN REVIEW					
I-12	fee_amount_Id Uses Narrower Integer Range	Informational	INFO	RESOLVED	p. 22
I-13	Document write handler authorization	Documentation	INFO	RESOLVED	p. 23
I-14	OFT TLV Metadata Misreports Account Layouts	Informational	INFO	RESOLVED	p. 24
I-15	Blacklist checks miss token account owners	Logical Error	INFO	ACKNOWLEDGED	p. 25
ROUND 2 - REMEDIATION REVIEW					
L-01	Quote Can Fail When Global Limits Are Disabled	DoS	LOW	RESOLVED	p. 26
L-02	Rate-Limit No-Op Cases Still Require Per-EID PDA	Unexpected Behavior	LOW	RESOLVED	p. 27
L-03	BurnMint Whitelist-Mode Escrow Send DoS	Compatibility	LOW	RESOLVED	p. 28
L-04	Missing Mint Authority Check During Initialize	Validation	LOW	ACKNOWLEDGED	p. 29
L-05	Enforced options msg_type mismatch	Configuration	LOW	RESOLVED	p. 30
I-01	Transfer Hook Index-3 Authority Doc Mislabeled	Documentation	INFO	RESOLVED	p. 31
I-02	Document Solana Compose Codec Layout	Documentation	INFO	RESOLVED	p. 32

§ 04

Findings, continued.

Index page 3 of 4.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
ROUND 2 - REMEDIATION REVIEW					
I-03	Shared Program Upgrades Affect All OFT Instances	Trust Assumptions	INFO	RESOLVED	p. 33
I-04	Dust Can Misalign Hook Accounts	Documentation	INFO	RESOLVED	p. 34
I-05	RoleType is always required	Informational	INFO	RESOLVED	p. 35
I-06	Receive-Types Token Mint Comment Mismatch	Informational	INFO	RESOLVED	p. 36
I-07	Mint Bypass Comment Names Wrong Modifier	Informational	INFO	RESOLVED	p. 37
I-08	Fees Should Account For Executor Funded ATA Creation	Documentation	INFO	RESOLVED	p. 38
I-09	Document Escrow Entries In Whitelist Mode	Documentation	INFO	RESOLVED	p. 39
I-10	QuoteOFT Max Understates Sendable Amount	Unexpected Behavior	INFO	RESOLVED	p. 40
I-11	recoverFunds Parity Comment Uses Wrong Signature	Documentation	INFO	RESOLVED	p. 41
I-12	Fund-Recovery Authority Rotation Undocumented	Documentation	INFO	RESOLVED	p. 42
I-13	Rate Exemption Seed Name Typo	Documentation	INFO	RESOLVED	p. 43

§ 04

Findings, continued.

Index page 4 of 4.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
ROUND 2 - REMEDIATION REVIEW					
I-14	Per-ID PDA Seed Width Docs	Documentation	INFO	RESOLVED	p. 44
I-15	Rate Limiter Docs Cite Wrong File	Documentation	INFO	RESOLVED	p. 45
I-16	Pause Docs Cite Wrong EVM Contract	Documentation	INFO	RESOLVED	p. 46
I-17	Misleading fee inverse edge cases	Documentation	INFO	RESOLVED	p. 47
I-18	Arbitrary compose plans expose Executor tokens	Validation	INFO	ACKNOWLEDGED	p. 48
ROUND 3 - REMEDIATION REVIEW					
L-01	Missing RateLimit PDA mismatch	Logical Error	LOW	RESOLVED	p. 50
I-01	Informational Note Regarding Rate Limit Test	Informational	INFO	RESOLVED	p. 51
I-02	Rate-limit PDA requirement is underdocumented	Documentation	INFO	RESOLVED	p. 52
I-03	Misleading Globally-Disabled Usage Doc	Documentation	INFO	RESOLVED	p. 53

INFO I-01

RESOLVED

Allowlist Mode ABI Uses Raw u8.

CATEGORY Compatibility

LOCATION apps/oft-app/contracts/solana/transfer-hook/macros/src/instructions/allowlist/set_allowlist_mode.rs#L7

REVIEW Round 1 - Main Review

DESCRIPTION

The transfer-hook interface currently defines `set_allowlist_mode` with a raw `u8` parameter, so the instruction ABI can carry any value between 0-255. However, the intended domain for allowlist mode is only three enum values: `Open`, `Blacklist`, and `Whitelist`. The framework includes `TryFrom<u8>` for `AllowlistMode`, which correctly constrains `u8` to those three values, but because the ABI type is still `u8`, downstream projects must remember to call `AllowlistMode::try_from` themselves in each relevant handler (for example in `initialize`). If validation is omitted or implemented incorrectly, invalid mode values can reach business logic and cause unexpected behaviors that differs from the intended allowlist policy.

RECOMMENDATION

Either explicitly document as a requirement that all implementations should validate mode inputs using `AllowlistMode::try_from`, or consider changing the instruction parameter type from `u8` to `AllowlistMode` so invalid values are rejected at the interface boundary by construction.

RESOLUTION

Status: Resolved.

INFO I-02

RESOLVED

Parity doc misdescribes rate_limits return type.

CATEGORY Documentation

LOCATION apps/oft-app/contracts/solana/oft-extended/macros/src/instructions/rate_limiter/rate_limits.rs#L13

REVIEW Round 1 - Main Review

DESCRIPTION

The parity doc (solana-evm-parity-analysis.md §4.1, §4.8) documents rate_limits as returning bare RateLimit , matching the EVM IRateLimiter.rateLimits(uint256) → RateLimit signature. The actual spec in oft-extended/macros/src/instructions/rate_limiter/rate_limits.rs returns Option<RateLimit> : return_type: quote! (Option<::oft_extended::types::RateLimit>), Option<RateLimit> is the correct Solana shape - per-ID rate-limit state lives in a PDA that may or may not exist (None = no PDA initialized for that EID, Some(v) = configured). Off-chain SDKs that bind from the EVM-parity doc rather than the IDL misdecode the response: the Option tag byte (0x00/0x01) is read as the first field of RateLimit (config.override_default_config: bool), producing a one-byte-misaligned decode for every rate_limits query.

In addition, on Solana the RateLimit struct fields are inverted - config is first, state is second. This is opposite on EVM. pub struct RateLimit { pub config: RateLimitConfig, pub state: RateLimitState, } The docs also present the RateLimit struct as if it's a flat struct, but in reality it's wrapping RateLimitConfig and RateLimitState

RECOMMENDATION

Update the doc (no code change). In §4.1 row 2, change the Solana Return column from RateLimit to Option<RateLimit> . In §4.8, prepend a paragraph using the same template as §1.5/§3.3 (" Some(v) = per-ID PDA exists; None = no PDA, fall back to default"). Also update the docs to reflect the fact that RateLimit is not flat. Consider if reversing the order is desired.

RESOLUTION

Status: Resolved.

INFO I-03

RESOLVED

FeeDepositSet doc names nonexistent EVM event.

CATEGORY Documentation

LOCATION apps/oft-app/contracts/solana/oft-extended/src/events.rs#L100

REVIEW Round 1 - Main Review

DESCRIPTION

The event doc in `oft-extended/src/events.rs` cites an EVM event named `FeeDepositUpdated`, which does not exist in the EVM interfaces: `/// Emitted when the fee deposit is updated. /// EVM: event FeeDepositUpdated(address indexed feeDeposit) #[event] pub struct FeeDepositSet { pub oft_store: Pubkey, pub fee_deposit: Pubkey, }` The actual EVM event declared in <https://github.com/LayerZero-Labs/monorepo-external/blob/main/contracts/common/utls/evm/non-upgradeable/contracts/interfaces/IFeeHandler.sol#L20> is `event FeeDepositSet(address indexed feeDeposit);`. The Solana event name (`FeeDepositSet`) matches the EVM event name exactly; only the doc comment misnames the EVM equivalent.

RECOMMENDATION

Update the doc comment to reference the actual EVM event: `- /// EVM: event FeeDepositUpdated(address indexed feeDeposit) + /// EVM: event FeeDepositSet(address indexed feeDeposit)`

RESOLUTION

Status: Resolved.

INFO I-04

RESOLVED

Parity doc references stale `scale_decimals` field.

CATEGORY Documentation

LOCATION `apps/oft-app/contracts/solana/oft-extended/src/types.rs#L58-L66`

REVIEW Round 1 - Main Review

DESCRIPTION

The parity doc (`solana-evm-parity-analysis.md` §4.2, §5.2) documents `RateLimitGlobalConfig` as `{ use_global_state, is_globally_disabled, scale_decimals: u8 }` and explicitly lists `scale_decimals` as a "Solana-only addition" that is "runtime-configurable" (vs. the EVM `SCALE_DECIMALS` immutable). The actual struct in `oft-extended/src/types.rs` ships only the two bools: `pub struct RateLimitGlobalConfig { pub use_global_state: bool, pub is_globally_disabled: bool, }` `SetRateLimitGlobalConfigParams` matches the same two-field shape, and no separate setter for `scale_decimals` exists among the 21 oft-extended instructions. Off-chain SDKs and integrators that bind from the EVM-parity doc misdecode `RateLimitGlobalConfig` (extra byte read into a non-existent field).

RECOMMENDATION

Consider deleting the `scale_decimals` references from the parity doc so it matches the code.

RESOLUTION

Status: Resolved.

INFO I-05

RESOLVED

OftInfo.oft_type raw u8 bypasses enum guard.

CATEGORY Best Practices LOCATION apps/oft-app/contracts/solana/oft/src/oft_info.rs#L44

REVIEW Round 1 - Main Review

DESCRIPTION

`OftType ( oft/src/oft_info.rs:19 )` is a two-variant enum with `#[borsh(use_discriminant = true)]`, which would reject any byte outside `{0, 1}` during deserialization. But `OftInfo ( oft_info.rs )` stores the custody model as a raw `u8`, not as `OftType`: `pub struct OftInfo { pub version: u8, pub oft_type: u8, // raw byte, not OftType pub instructions: OftInstructionVersions, pub project_discriminator: [u8; 8], }` All fields are `pub`. The safe constructor `OftInfo::new(oft_type: OftType, ...)` takes the enum and casts it (`oft_type as u8`), but a caller can also build the struct directly (`OftInfo { oft_type: 99, ... }`) without a compile-time check or discriminant guard on read. Downstream consumers pull a raw `u8` and must validate themselves.

RECOMMENDATION

Type the field as the enum so the Borsh guard applies on both write and read paths: `pub struct OftInfo { pub version: u8, pub oft_type: OftType, pub instructions: OftInstructionVersions, pub project_discriminator: [u8; 8], }`

RESOLUTION

Status: Resolved.

INFO I-06

RESOLVED

Undocumented RoleType requirement in #[oft].

CATEGORY Documentation

LOCATION apps/oft-app/contracts/solana/oft/macros/src/oft.rs#L46

REVIEW Round 1 - Main Review

DESCRIPTION

The `#[oft]` macro injects `#[::oapp::oapp(state = #oft_type, role_type = crate::RoleType)]` (`oft/macros/src/oft.rs:46`). The `role_type` path is hardcoded - consumers must declare a `RoleType` enum at crate root with that exact name. `#[oft_extended]` inherits this via its nested `#[oft]` injection. This implicit requirement isn't mentioned in any documentation. Consumers only discover it via compile errors.

RECOMMENDATION

Add documentation stating that consumers must declare a `RoleType` enum at crate root satisfying the `rbac` derive contract.

RESOLUTION

Status: Resolved.

INFO I-07

RESOLVED

Rate limiter doc names nonexistent EVM file.

CATEGORY Documentation

LOCATION apps/oft-app/contracts/solana/oft-extended/src/events.rs#L56

REVIEW Round 1 - Main Review

DESCRIPTION

The events section in oft-extended/src/events.rs claims EVM alignment with a file that does not exist: `// ===== Rate Limiter Events ===== // EVM alignment: IRateLimiterCore.sol` No IRateLimiterCore.sol exists anywhere. The actual EVM interface declaring all four rate-limiter events in this section `( RateLimitConfigUpdated , RateLimitGlobalConfigUpdated , RateLimitStateUpdated , RateLimitAddressExemptionUpdated )` is IRateLimiter.sol .

RECOMMENDATION

Update the comment to reference the actual EVM interface file.

RESOLUTION

Status: Resolved.

INFO I-08

RESOLVED

getRateLimitUsages doc misnames EVM params.

CATEGORY Documentation

LOCATION apps/oft-app/contracts/solana/oft-extended/macros/src/instructions/rate_limiter/get_rate_limit_usages.rs#L3-L5

REVIEW Round 1 - Main Review

DESCRIPTION

The macro doc in `get_rate_limit_usages.rs` cites the EVM return-parameter names incorrectly: `//! EVM: IRateLimiter.getRateLimitUsages(uint256 _id) → //! (uint256 outboundUsage, uint256 outboundAvailable, uint256 inboundUsage, uint256 //! inboundAvailable)` The actual EVM signature declared in `IRateLimiter.sol` is: `function getRateLimitUsages( uint256 _id ) external view returns ( uint256 outboundUsage, uint256 outboundAvailableAmount, uint256 inboundUsage, uint256 inboundAvailableAmount );`

RECOMMENDATION

Update the doc comment to reference the actual EVM return parameter names.

RESOLUTION

Status: Resolved.

INFO I-09

ACKNOWLEDGED

Unqualified Pubkey Makes Expansion Import-Sensitive.

CATEGORY Compatibility

LOCATION apps/oft-app/contracts/solana/oft/macros/src/instructions/token.rs#L13 and related helpers

REVIEW Round 1 - Main Review

DESCRIPTION

Several OFT / OFT-extended instruction specs emit bare `Pubkey` types in generated signatures instead of fully qualified paths. As a result, macro expansion depends on the downstream module already importing `Pubkey` into scope, so otherwise correct implementations may fail to compile for reasons unrelated to their business logic. While this is a compile-time developer footgun rather than a runtime issue, it reduces the robustness and portability of these framework macros for third-party consumers.

RECOMMENDATION

Emit fully qualified paths for generated public types, such as `::anchor_lang::prelude::Pubkey`, so macro expansion is independent of downstream imports.

RESOLUTION

Status: Acknowledged.

The current design was retained because using the fully qualified type breaks Anchor IDL generation; consumers still need to import `Pubkey`.

INFO I-10

RESOLVED

Inconsistent OFT type docs.

CATEGORY Documentation

LOCATION apps/oft-app/contracts/solana/oft/src/types.rs#L40 apps/oft-app/contracts/solana/oft/src/types.rs#L104

REVIEW Round 1 - Main Review

DESCRIPTION

The `///` documentation for `QuoteOFTResult` states that it provides the fee breakdown, limits, and receipt data, but both the struct field order and the documented `IOTF.quoteOFT()` return tuple order are `OFTLimit`, `OFTFeeDetail[]`, and `OFTReceipt`. The `///` documentation for `QuoteOFTParams` states that it provides the fee breakdown and settings data for an OFT, but `QuoteOFTParams` is the input parameter object used by `quoteOFT()`. The fee breakdown and receipt-related data are returned through `QuoteOFTResult`, not provided by the params type.

RECOMMENDATION

Update the `QuoteOFTResult` documentation to list the returned data in the same order as the struct and `IOTF.quoteOFT()` return tuple: limits, fee breakdown, and receipt data. Make the documentation describe the struct as the parameters used to quote an OFT transfer, rather than as data that provides fee breakdown or settings information.

RESOLUTION

Status: Resolved.

INFO I-11

RESOLVED

Misleading oft_app comment.

CATEGORY Documentation

LOCATION apps/oft-app/contracts/solana/oft/src/oft_info.rs#L34-L35 apps/oft-app/contracts/solana/oft/src/oft_info.rs#L48-L49

REVIEW Round 1 - Main Review

DESCRIPTION

There are 2 comments about `OftInfo` which suggest it's possible for an SDK to jump directly to a TLV entry without scanning the buffer thanks to the `OAppBase.app_discriminator` and `OftInfo.project_discriminator`. This is misleading, since the discriminators are used for type discovery, but they cannot serve the goal of direct jump, as there is no offset encoded in them.

RECOMMENDATION

Clear up the comments.

RESOLUTION

Status: Resolved.

INFO I-12

RESOLVED

fee_amount_ld Uses Narrower Integer Range.

CATEGORY Informational LOCATION apps/oft-app/contracts/solana/oft/src/types.rs#L34

REVIEW Round 1 - Main Review

DESCRIPTION

`OFTFeeDetail.fee_amount_ld` is defined as `i64`, while the broader OFT amount model uses `u64`. As a result, `fee_amount_ld` supports a narrower value range than other token amount fields in the interface. Although fee values are unlikely to approach these limits in typical Solana deployments, the reduced range may still be relevant for integrations or tooling that assume fee metadata can represent the full token amount domain.

RECOMMENDATION

Consider using a signed type that fully covers the `u64` amount domain, such as `i128`, or explicitly documenting that `fee_amount_ld` is intentionally narrower as part of the interface design.

RESOLUTION

Status: Resolved.

INFO I-13

RESOLVED

Document write handler authorization.

CATEGORY Documentation

LOCATION oft, oft_extended

REVIEW Round 1 - Main Review

DESCRIPTION

The `oft_extended` documentation states that required instructions such as `set_rate_limit_state()` have no default implementation and must be implemented by concrete OFT programs, but it does not clearly state that write handlers must also implement authorization matching the EVM RBAC surface. This is easy to miss because the macro only enforces that a handler exists; it does not enforce that the handler includes `#[rbac::only_role(...)]` or another access-control check.

RECOMMENDATION

Document that required OFT Extended write handlers must add authorization explicitly, because the macro does not provide default RBAC.

RESOLUTION

Status: Resolved.

INFO I-14

RESOLVED

OFT TLV Metadata Misreports Account Layouts.

CATEGORY Informational

LOCATION oft_info.rs

REVIEW Round 1 - Main Review

DESCRIPTION

OftInfo tells SDKs that all base OFT instructions use standard account layout version 1, but the OFT framework does not define or enforce any standard account layout for those instructions. All five OFT instructions are required overrides. Each concrete program chooses its own `Context<T>` account layout, yet the shared metadata still advertises them as standard layouts. This can cause generic SDKs or executors that trust `OAppInfo` / `OftInfo` to build the wrong account list, making core OFT actions like `send`, `quote_send`, and `quote_of` fail for users. `OftInfo` hardcodes every base OFT instruction version to 1 in

`OftInstructionVersions` `OftInfo::new()` then publishes those versions during initialization. But the OFT macro does not provide predefined account templates for these instructions, every OFT instruction has `default_impl: None`

So the concrete program must implement the handler, and `anchor-trait` dispatches using the handler declared `Context<T>`

Example `#[oft_instruction] pub fn send( ctx: &mut Context<MyCustomSendAccounts>, params: &SendParams, ) → Result<(MessagingReceipt, OFTReceipt)> { }` The metadata says `send = 1`, but the real account layout is

`MyCustomSendAccounts`. In `console_of` for example, the mismatch is real `send` requires `pause`, `fee`, `rate-limit`, `exemption`, `token source`, `escrow`, `fee deposit`, `mint`, `token program`, and `Endpoint` accounts. Those are concrete program specific layouts, not framework generated OFT standard layouts. A metadata driven SDK can read `OftInfo.instructions.send = 1` and take the standard discovery path, even though no standard `send` layout exists.

Impact is SDK built `send` / `quote_send` transactions can omit required concrete accounts. Routes may become unusable for users relying on generic account discovery. Note: If a project has generated SDK from its IDL, that SDK knows the real custom accounts, and that is safe in that case. But the on chain metadata still publishes `OftInfo.instructions.send = 1`. So any generic OFT / `OApp` metadata driven resolver that does not use the project / app IDL SDK, and instead trusts `OAppInfo` / `OftInfo`, can still build the wrong account list.

RECOMMENDATION

Set OFT instruction versions to 0, 1 is only correct if the framework actually defines OFT base instruction account layout v1, and currently it does not.

RESOLUTION

Status: Resolved.

INFO I-15

ACKNOWLEDGED

Blacklist checks miss token account owners.

CATEGORY Logical Error

LOCATION Transfer-hook

REVIEW Round 1 - Main Review

DESCRIPTION

During Token-2022 transfer-hook execution, the concrete transfer checks are described as covering "source/destination token-account PDAs plus signer/delegate" That omits the most important identities for blacklist enforcement

`source_token_account.owner` `destination_token_account.owner` In SPL Token / Token-2022, the destination account passed to a transfer is a token account, not necessarily the user wallet. The wallet owner is stored inside the token account data. A user can create many token accounts, and those token account pubkeys will not equal the user wallet pubkey that was blacklisted Similarly, when a delegate transfers on behalf of a blacklisted user, the transfer authority / signing account is the delegate, not the token account owner. If only the delegate / signer is checked, a blacklisted owner can move funds through a clean delegate

So blacklist mode can be bypassed by any blacklisted user who creates new token accounts or uses a non blacklisted delegate, This defeats the stated policy that listed addresses are blocked

RECOMMENDATION

In the Token-2022 transfer_hook, deserialize the source and destination token accounts and enforce blacklist / whitelist policy against their owner fields. blacklist mode should reject if any of these identities are blacklisted `source_token_account.owner` `destination_token_account.owner` `transfer_authority_or_delegate` .

RESOLUTION

Status: Acknowledged.

The current token-account-based allowlist design was retained because wallet-owner checks would break account resolution for first-time recipient ATAs.

LOW **L-01**

RESOLVED

Quote Can Fail When Global Limits Are Disabled.

CATEGORY DoS

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/state/rate_limit.rs#L219-L227

REVIEW Round 2 - Remediation Review

DESCRIPTION

Solana's `get_rate_limit_usages` follows the same high-level order as the EVM implementation: it resolves the effective rate-limit state/config, computes the current usage, and only then applies the global-disable override by returning maximum available capacity. On EVM, this ordering is safe because `rateLimits[id]` is a mapping lookup and always returns a default `RateLimit` struct even when the ID was never explicitly initialized.

On Solana, the same ordering has different behavior because per-EID rate limits are optional PDAs. When `use_global_state = false`, `get_rate_limit_usages` must load the per-EID PDA before it can compute usage. If that PDA is missing, the function returns `RateLimitNotInitialized` before reaching the global-disable branch, even when `is_globally_disabled = true`. Therefore, two implementations with the same logical ordering diverge because Solana has a missing-account failure mode that the EVM mapping model does not. This impacts more than the standalone rate-limit view.

Solana's `quote_offt` uses `get_rate_limit_usages` to compute `max_amount_ld`. Integrators that call `quote_offt` before `send` to obtain OFT limits, fee breakdown, and the expected receipt can fail unexpectedly even though `send` would succeed, because the mutating rate-limit path returns early when global rate limiting is disabled.

RECOMMENDATION

Make `get_rate_limit_usages` return maximum quote/view availability when global rate limiting is disabled before treating a missing optional per-EID PDA as an error.

RESOLUTION

Status: Resolved.

LOW **L-02**

RESOLVED

Rate-Limit No-Op Cases Still Require Per-EID PDA.

CATEGORY Unexpected Behavior

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/state/rate_limit.rs#L321-L355

REVIEW Round 2 - Remediation Review

DESCRIPTION

Solana represents per-EID rate-limit state as an optional PDA, while EVM uses a mapping whose missing entries read as a zero-initialized struct. On Solana, `apply_rate_limit` returns early only when `is_globally_disabled` is true; for every other case it calls `get_effective_state_and_config_mut`, which returns `RateLimitNotInitialized` when `use_global_state = false` and the per-EID PDA is uninitialized. The two no-op gates the limiter actually honors, an effective direction disabled by the default config and an address-exempt user, are checked only after that resolution. So even when the config disables the relevant direction or the caller is exempt, a `send` or `lz_receive` for an EID whose per-EID `RateLimit` PDA was never created reverts with `RateLimitNotInitialized` before reaching the no-op gate. The read path has the same shape: `get_rate_limit_usages` resolves state and config first and only then applies the disabled-direction availability override, so `quote_oft` reverts for the same EIDs even though the effective answer is unlimited.

On EVM, `_getRateLimitStateAndConfig` reads `rateLimits[id]` directly from a mapping, so an uninitialized ID yields a zero state plus the default config and never reverts. The disabled-direction and address-exemption early returns are honored without any per-ID state ever being written. The Solana account model therefore introduces a missing-account failure mode for cases that EVM treats as unconditional no-ops. This is a Solana-only liveness footgun: the limiter is configured to allow the operation, but the call fails until the operator separately initializes per-EID state.

RECOMMENDATION

Evaluate the no-op conditions (disabled direction, address exemption) against the default config before treating a missing optional per-EID PDA as an error, or have the resolver fall back to default config and zeroed state to match EVM mapping semantics.

RESOLUTION

Status: Resolved.

LOW **L-03**

RESOLVED

BurnMint Whitelist-Mode Escrow Send DoS.

CATEGORY Compatibility

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/instructions/send.rs#L240

REVIEW Round 2 - Remediation Review

DESCRIPTION

On a BurnMint OFT `send`, `console_oft` always routes the full amount from the sender to the escrow before burning. `lock_or_burn` issues `invoke_transfer_checked(token_source → token_escrow, amount_sent_ld)` and only then burns from the escrow. That `user → escrow` leg is a Token-2022 `transfer_checked`, so it fires the transfer hook with source = sender ATA, destination = escrow ATA, and authority = sender signer, and the hook runs the full source plus destination plus authority allowlist check. In Whitelist mode `AllowlistMode::allows` returns the whitelist flag, so the escrow ATA, as the transfer destination, must itself be whitelisted or the hook returns `DestinationBlocked` and the send reverts.

The escrow source-bypass does not rescue this leg. The bypass marker is keyed off the transfer source, but on send the escrow is the destination, not the source, which the hook comment in `transfer_hook.rs` confirms. The net effect is that enabling Whitelist mode silently bricks all BurnMint outbound sends until an operator separately whitelists the escrow token account. The condition is operator-introduced and recoverable by whitelisting the escrow, but it is easy to overlook because the requirement is undocumented.

This requirement has no EVM analogue. On EVM, BurnMint `_debit` burns the amount directly from the user via `burn(_from, amountSentLD)` with no intermediate vault, and `ERC20Plus.burn` checks only `onlyAllowlisted(_from)`; the fee is minted to `feeDeposit` via `mint()`, which carries no allowlist modifier. EVM BurnMint therefore needs only the user whitelisted, there is no escrow or vault to whitelist. The Solana-only escrow-routing design introduces the extra destination gate. The EVM-parity documentation does not surface this and in fact contradicts it: `transfer-hook-evm-parity-analysis.md` labels the BurnMint send path "Equivalent for OFT-controlled burn path," describing it only as "checks source before burn" and omitting the destination and authority checks that the same hook performs. An operator porting a BurnMint deployment from EVM, or a reviewer relying on the parity table, would have no indication that the escrow must be whitelisted, so every outbound `send` reverts with `DestinationBlocked` until the escrow is explicitly whitelisted.

RECOMMENDATION

Auto-whitelist the escrow token account when Whitelist mode is enabled. At minimum, document the requirement and correct the `transfer-hook-evm-parity-analysis.md` row so it no longer labels the BurnMint send path "Equivalent" without noting that the escrow destination check is a Solana-only gate absent from EVM `burn`.

RESOLUTION

Status: Resolved.

LOW **L-04**

ACKNOWLEDGED

Missing Mint Authority Check During Initialize.

CATEGORY Validation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/instructions/init_console_oft.rs#L52-L56

REVIEW Round 2 - Remediation Review

DESCRIPTION

`init_console_oft` allows an OFTStore to be initialized with `oft_type = BurnMint` without validating that the OFTStore can actually mint the underlying token. The inbound `lz_receive` path later requires the token mint authority to be either the `oft_store` PDA itself or a compatible Token-2022 multisig with quorum 1 that includes `oft_store` as a signer. If the mint authority is an admin wallet, an incompatible multisig, or another authority the OFTStore cannot satisfy, initialization still succeeds but inbound delivery fails when the program attempts to mint.

This can leave a deployment in a difficult or permanent failure state. The setup is only recoverable while the current mint authority remains controllable and can be changed through the token program. If the mint authority is disabled or cannot be reassigned to a compatible authority, the BurnMint OFT instance remains unable to receive inbound messages.

RECOMMENDATION

During initialization of the BurnMint type, require proof that the mint authority is either `oft_store` or a compatible 1-of-n Token-2022 multisig containing `oft_store`

RESOLUTION

Status: Acknowledged.

LOW **L-05**

RESOLVED

Enforced options msg_type mismatch.

CATEGORY Configuration

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/helpers/mod.rs#L10-L14

REVIEW Round 2 - Remediation Review

DESCRIPTION

The Solana OFT implementation defines `MsgType::Send = 0` and `MsgType::SendAndCall = 1`, while LayerZero's public FAQ and EVM OFT implementation use `msgType = 1` for `SEND` and `msgType = 2` for `SEND_AND_CALL`. The `send()` path derives the `EnforcedOptions` PDA from the local `msg_type()` result. If an operator follows the documented 1/2 convention or uses shared tooling that assumes EVM constants, they can configure `msgType = 1` for plain sends and `msgType = 2` for compose sends. Plain Solana sends then look for the unconfigured `msgType = 0` PDA; when it is missing, `try_load_account()` returns `None`, and `combine_options()` falls back to empty enforced options. As a result, the message can proceed using only caller-provided `extra_options`. Similarly, packets with compose messages will use the `SEND` options and the `SEND_AND_CALL` options will be unused.

This misconfiguration is also easier to introduce because `SetEnforcedOptionsParams::msg_type` accepts raw numeric values as its type is `u16`.

RECOMMENDATION

Either make the SVM values 1/2 as well, or explicitly document that on Solana 0/1 should be used and consider overriding `set_enforced_options` with additional validation.

RESOLUTION

Status: Resolved.

INFO I-01

RESOLVED

Transfer Hook Index-3 Authority Doc Mismatch.

CATEGORY Documentation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_transfer_hook/src/instructions/init_transfer_hook.rs#L127

REVIEW Round 2 - Remediation Review

DESCRIPTION

The account-index documentation table in `init_transfer_hook.rs` labels index 3 as `authority / owner`. This is inaccurate: index 3 is the transfer authority that Token-2022 passes into the `Execute` hook, which may be the source owner, an approved delegate, or the mint's permanent delegate. The `owner` qualifier captures only one of the three cases and is misleading for the delegate and permanent-delegate recovery paths. The authoritative description in `transfer_hook.rs` correctly documents index 3 as `authority (source owner, approved delegate, or permanent delegate)`, so the two canonical index tables disagree with each other.

The runtime constant `AUTHORITY_ACCOUNT_INDEX` is correct; only the doc row is wrong, so there is no behavioral impact, but a reader relying on the `init` table could misreason about the PD fund-recovery and delegated-transfer flows that key the `authority_allowlist_entry` PDA off this account.

RECOMMENDATION

Update the index-3 row in the `init_transfer_hook.rs` table to match `transfer_hook.rs`. Use either `authority` alone or the fully-qualified `authority (owner / delegate / permanent delegate)` so the two tables are consistent and the delegate / permanent-delegate cases are not obscured.

RESOLUTION

Status: Resolved.

INFO I-02

RESOLVED

Document Solana Compose Codec Layout.

CATEGORY Documentation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/codec/compose_msg.rs#L4-L23 and related helpers

REVIEW Round 2 - Remediation Review

DESCRIPTION

The Solana compose codec comment says it is aligned with `OFTComposeMsgCodec`, but the implementation encodes `amount_ld` as a `u64` occupying 8 bytes. Its compose header is therefore `[nonce:8][src_eid:4][amount_ld:8]`, with `compose_from` at offset 20 and user compose payload at offset 52. The EVM OFT compose codec encodes `amountLD` as `uint256` in `abi.encodePacked`, so the header is `[nonce:8][srcEid:4][amountLD:32]`, with `composeFrom` at offset 44 and user compose payload at offset 76. If the compact 8-byte Solana layout is intentional, the issue is the comment/documentation rather than the codec behavior. The current wording can make readers assume the Solana payload is byte-compatible with the EVM `OFTComposeMsgCodec` layout even though the offsets differ.

RECOMMENDATION

Update the codec comment and related docs to state that the Solana compose payload uses a Solana-specific compact layout with an 8-byte `amount_ld`, `COMPOSE_FROM_OFFSET = 20`, and `COMPOSE_MSG_OFFSET = 52`.

RESOLUTION

Status: Resolved.

INFO I-03

RESOLVED

Shared Program Upgrades Affect All OFT Instances.

CATEGORY Trust Assumptions

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/lib.rs#L37-L43

REVIEW Round 2 - Remediation Review

DESCRIPTION

Solana `console_oft` can host multiple OFTStore instances under one deployed program ID. Although each instance has separate state and admin configuration, all instances execute the same program binary. If that program remains upgradeable, a later upgrade changes behavior for existing and future OFTStore instances. This differs from EVM, where developers deploy separate OFT proxy instances from the LayerZero implementation contracts. Upgrading one EVM proxy does not automatically affect other OFTs, while upgrading a shared Solana program affects every OFTStore under that program ID. This is an implicit trust assumption for developers using a shared deployment.

RECOMMENDATION

Document the shared upgrade-authority trust model and advise production deployers to self-deploy, use governance-controlled upgrades, or freeze the program when appropriate.

RESOLUTION

Status: Resolved.

INFO I-04

RESOLVED

Dust Can Misalign Hook Accounts.

CATEGORY Documentation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/instructions/send.rs#L284-L290

REVIEW Round 2 - Remediation Review

DESCRIPTION

`send` decides whether to run the escrow $\rightarrow$ fee deposit transfer from `fee_ld = amount_sent_ld - amount_received_ld`, not from the configured fee bps. Since `amount_received_ld` is rounded down by `remove_dust`, `fee_ld` includes both configured OFT fee and the dust. A send with zero fee bps can therefore still enter `collect_fee` when `amount_ld` is not aligned to `ld2sd_rate`.

For Token-2022 mints with an active transfer hook, `collect_fee` expects a second hook-account prefix before the Endpoint accounts. Callers that assemble `remaining_accounts` from the configured OFT fee can omit this second hook prefix for dust-only fees, causing the program to parse the Endpoint account as the fee hook program and revert with `InvalidTransferHookProgram`. The repo's test helper also reflects this assumption by computing `hasFee` from `amount * bps / 10000`, while the on-chain condition is the dust-inclusive receipt delta. `// Fee hook accounts are only included when fees will be collected (fee_ld > 0). // The on-chain program splits remaining_accounts as: // [transfer_hook..., fee_hook...(if fee>0), endpoint...] // so including fee hooks when fee=0 would misalign the endpoint accounts slice.`

`// Must compute fee_ld using the same integer arithmetic as on-chain (amount * bps / 10000).``

RECOMMENDATION

Document that fee-hook accounts must be included whenever `amount_sent_ld - amount_received_ld > 0`, including dust-only deltas, not only when configured fee bps produces a nonzero fee.

RESOLUTION

Status: Resolved.

INFO I-05

RESOLVED

RoleType is always required.

CATEGORY Informational LOCATION apps/oft-app/contracts/solana/oft/macros/src/oft.rs#L49

REVIEW Round 2 - Remediation Review

DESCRIPTION

The [I-06] Undocumented `RoleType` requirement in `#[oft]` was resolved with the following comment that says: `#[oft]` now [...] accepts an optional `role_type = attribute`, defaulting to `crate::RoleType` when omitted. The actual implementation doesn't have such fallback logic, instead the `role_type` is required.

RECOMMENDATION

Confirm whether the field should be required or optional.

RESOLUTION

Status: Resolved.

INFO I-06

RESOLVED

Receive-Types Token Mint Comment Mismatch.

CATEGORY Informational

LOCATION `apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/instructions/lz_receive_types_info.rs` and related helpers

REVIEW Round 2 - Remediation Review

DESCRIPTION

The step-1 receive-types comment says `token_mint` is included because V2 needs it to "derive token_program for ATA creation". The implementation uses the token program stored in `oft_store.token_program`, not a token program derived from `token_mint`. `lz_receive_types_info` derives the destination ATA with `ctx.accounts.oft_store.token_program` and passes that stored token program as its own required account. `lz_receive_types_v2` then receives `token_program` as a separate account and uses it in the token-mint constraint and destination ATA derivation. The `token_mint` account is still needed by V2, but for mint validation and mint-data reads such as mint authority, decimals, and transfer-hook extension metadata. The current comment points readers to the wrong dependency when reviewing receive-account planning.

RECOMMENDATION

Update the comment to explain that `token_mint` is included so V2 can validate and read the mint, including transfer-hook metadata, while `token_program` comes from `oft_store.token_program`.

RESOLUTION

Status: Resolved.

INFO I-07

RESOLVED

Mint Bypass Comment Names Wrong Modifier.

CATEGORY Informational

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_transfer_hook/src/instructions/transfer_hook.rs#L16-L20

REVIEW Round 2 - Remediation Review

DESCRIPTION

The transfer-hook module comment says EVM `mint()` has neither `whenNotGloballyPaused` nor `onlyAllowlisted` modifiers. In the scoped EVM implementation, the pause modifier used by `ERC20Plus` is `whenNotPaused`; `mint()` is gated only by `onlyRole(MINTER_ROLE)`, while `burn`, `transfer`, and `transferFrom` use `whenNotPaused` and/or `allowlist` modifiers. The high-level statement that `mint()` bypasses pause/list checks is correct, but the modifier name does not match the EVM contract. This makes the Solana comment harder to verify against the referenced EVM behavior.

RECOMMENDATION

Change the comment to say that EVM `mint()` has neither `whenNotPaused` nor `onlyAllowlisted` modifiers.

RESOLUTION

Status: Resolved.

INFO I-08

RESOLVED

Fees Should Account For Executor Funded ATA Creation.

CATEGORY Documentation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/instructions/lz_receive.rs#L80-L81

REVIEW Round 2 - Remediation Review

DESCRIPTION

`lz_receive` creates the recipient token account with `init_if_needed` and uses the executor as payer for rent. The recipient address is taken from the delivered OFT message, so messages to first-time recipients can require the executor to fund a new ATA during destination execution. `quote_send` quotes the LayerZero endpoint fee from the message and options, but the on-chain quote path does not itself check whether the destination ATA already exists or whether rent will be needed. Integrators should ensure that the source-chain quoted/executor fees and execution options are sufficient to cover destination-side ATA creation and rent when the recipient token account may not already exist.

RECOMMENDATION

Document that first-time recipient ATA creation is funded during `lz_receive` by the executor payer, and that source-side fee quoting/options should account for this destination rent cost.

RESOLUTION

Status: Resolved.

INFO I-09

RESOLVED

Document Escrow Entries In Whitelist Mode.

CATEGORY Documentation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_transfer_hook/src/instructions/transfer_hook.rs#L165-L187

REVIEW Round 2 - Remediation Review

DESCRIPTION

In whitelist mode, an OFT escrow needs both a whitelist entry and a source-bypass entry for the intended bridge flow. Outbound sends transfer from the user to the escrow, so the escrow is checked as the destination and must be whitelisted. Inbound receives transfer from the escrow to the recipient, so the escrow is the source and needs source-bypass to avoid requiring every recipient or authority to be whitelisted. The `PermanentDeLegate` recovery path follows EVM parity and determines recovery eligibility from allowlist status, not source-bypass status. If the escrow is de-whitelisted while source-bypass remains enabled, the `PermanentDeLegate` can move funds from the escrow through the recovery path, even though source-bypass still applies to normal escrow-as-source transfers. OFT admins should be aware of the misconfiguration risk of source-bypassed but de-whitelisted escrows.

RECOMMENDATION

Document that whitelist-mode OFT escrows require both whitelist and source-bypass entries, and the risk of de-whitelisting a source-bypassed escrow.

RESOLUTION

Status: Resolved.

INFO I-10

RESOLVED

QuoteOFT Max Understates Sendable Amount.

CATEGORY Unexpected Behavior

LOCATION console_of/src/instructions/quote_of.rs(QuoteOFT::apply) and related helpers

REVIEW Round 2 - Remediation Review

DESCRIPTION

quote_of derives oft_limit.max_amount_ld by taking the outbound rate-limit availability and passing it through get_amount_before_fee: `// quote_of.rs if max_amount_ld != 0 && max_amount_ld != u64::MAX { max_amount_ld = fee_config::get_amount_before_fee( ctx.accounts.of_store.default_fee_bps, fee_config.as_ref(), max_amount_ld, ); } let oft_limit = OFTLimit { min_amount_ld: 0, max_amount_ld }; get_amount_before_fee` inverts only the fee and does nothing about dust: `// fee_config_view.rs pub fn get_amount_before_fee(default_fee_bps: u16, cfg: Option<&FeeConfig>, amount_after_fee: u64) -> u64 { let bps = get_fee_bps(default_fee_bps, cfg) as u128; let denom = MAX_FEE_BASIS_POINTS as u128; if bps >= denom { return 0; } if bps == 0 { return amount_after_fee; } let amount_before_fee = (amount_after_fee as u128) * denom / (denom - bps); u64::try_from(amount_before_fee).unwrap_or(u64::MAX)`

} The actual send path instead applies dust removal after the fee, and the rate limiter is charged against the dust-removed received amount: `// send.rs (debit_view) let oft_fee = fee_config::get_of_fee(of_store.default_fee_bps, fee_config, amount_ld); let amount_received_ld = of_store.remove_dust(amount_ld - oft_fee); // remove_dust(x) = x - x % ld2sd_rate` Because remove_dust maps a contiguous band of pre-fee amounts onto the same post-fee, dust-aligned received amount, several distinct pre-fee amount_ld values all produce the identical amount_received_ld. The fee-only inverse returns the smallest such pre-fee amount, so max_amount_ld lands one dust plateau short of the true maximum that fits within capacity.

The quote can therefore simultaneously report (a) that a specific requested send receives an amount within capacity and executes successfully, and (b) a max_amount_ld lower than that same requested amount. The under-report is always in the conservative direction, since the quote never advertises a max larger than what the send path accepts, so no rate-limit ceiling can be bypassed through it. Clients that trust oft_limit.max_amount_ld as a hard cap can reject sends that the program would accept, degrading the quote and view API's usefulness at the dust margin near the rate-limit ceiling. This is also the behavior on EVM, so if the decision is to fix this, it should also be fixed on EVM,

RECOMMENDATION

When converting post-fee capacity back to a max pre-fee send amount, account for the dust plateau by returning the largest pre-fee amount_ld whose remove_dust(amount_ld - fee(amount_ld)) still fits within capacity, rather than the fee-only inverse. Or clearly document this mismatch and acknowledge it as intended.

RESOLUTION

Status: Resolved.

INFO I-11

RESOLVED

recoverFunds Parity Comment Uses Wrong Signature.

CATEGORY Documentation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_transfer_hook/src/instructions/transfer_hook.rs

REVIEW Round 2 - Remediation Review

DESCRIPTION

The transfer-hook doc comments cite the EVM parity as `recoverFunds(from, amount)`. The actual EVM signature is `recoverFunds(address _from, address _to, uint256 _amount)` (ERC20Plus.sol#L144), the `_to` recipient argument is omitted.

RECOMMENDATION

Update the comment to `recoverFunds(from, to, amount)`.

RESOLUTION

Status: Resolved.

INFO I-12

RESOLVED

Fund-Recovery Authority Rotation Undocumented.

CATEGORY Documentation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_transfer_hook/src/instructions/transfer_hook.rs

REVIEW Round 2 - Remediation Review

DESCRIPTION

Blocked-source fund recovery is gated solely on the mint's Token-2022 PermanentDelegate (`get_permanent_delegate(&mint_data) ≠ Some(signer)` in `try_bypass_for_fund_recovery`), decoupled from the hook's RBAC. The docs already disclose this and label it intentional ("authority-model adaptation", "not bugs") in `transfer-hook-evm-parity-analysis.md` , `transfer-hook.md` , `AUDIT.md` , and the `transfer_hook.rs` module comment. Two consequences are NOT documented: (1) the PermanentDelegate rotates via a one-step SPL `SetAuthority` call with no pending/accept step, unlike EVM's two-step `DEFAULT_ADMIN_ROLE` handoff; (2) recovery keys off a principal distinct from the hook's `DefaultAdmin`, so it sits outside the governance protecting every other hook action. The capability is EVM-equivalent, so this is purely a documentation gap, not a privilege bug.

RECOMMENDATION

Note in `transfer-hook-evm-parity-analysis.md` that the recovery authority is the mint PermanentDelegate (a distinct principal from the hook `DefaultAdmin`) and rotates one-step via `SetAuthority` with no accept step.

RESOLUTION

Status: Resolved.

Rate Exemption Seed Name Typo.

CATEGORY Documentation

LOCATION apps/project-types/console-of-t-app/contracts/solana/programs/console_of_t/src/state/rate_limit.rs#L35 and related helpers

REVIEW Round 2 - Remediation Review

DESCRIPTION

The `RateLimitAddressExemption` PDA comment documents the seed as `RATE_LIMITER_EXEMPTION_SEED`. No constant with that name exists in the codebase. The actual seed constant is `RATE_LIMIT_EXEMPTION_SEED` in `seeds.rs`, and `RateLimitAddressExemption::seeds` uses that constant. The seed bytes are correct at runtime, but the comment points integrators and reviewers to a non-existent symbol.

RECOMMENDATION

Update the `RateLimitAddressExemption` PDA comment to reference `RATE_LIMIT_EXEMPTION_SEED`.

RESOLUTION

Status: Resolved.

INFO I-14

RESOLVED

Per-ID PDA Seed Width Docs.

CATEGORY Documentation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/state/rate_limit.rs#L16 and related helpers

REVIEW Round 2 - Remediation Review

DESCRIPTION

The `RateLimit`, `PauseConfig`, and `FeeConfig` PDA comments document the id seed as `&eid.to_be_bytes()`, which suggests the 4-byte big-endian EID encoding. The actual seed helpers use `id.to_u128_be_bytes()`, producing a 16-byte big-endian `u128` seed for rate-limit, pause, and fee PDAs. An SDK author or reviewer copying the comments literally would derive different non-resolving PDA addresses.

RECOMMENDATION

Update the three PDA seed comments to state that the id component is the canonical 16-byte big-endian `u128` encoding, matching `id.to_u128_be_bytes()`.

RESOLUTION

Status: Resolved.

INFO I-15

RESOLVED

Rate Limiter Docs Cite Wrong File.

CATEGORY Documentation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/state/rate_limit

REVIEW Round 2 - Remediation Review

DESCRIPTION

Several rate-limiter comments cite `RateLimiterCoreUpgradeable.sol` as the EVM reference for `_getRateLimitUsage`, `_getRateLimitUsages`, and `_getRateLimitStateAndConfig`. That file does not exist in the scoped EVM sources. The referenced functions live in `RateLimiterBaseUpgradeable.sol`, which the same Solana file already cites correctly elsewhere.

RECOMMENDATION

Replace the `RateLimiterCoreUpgradeable.sol` references with `RateLimiterBaseUpgradeable.sol`.

RESOLUTION

Status: Resolved.

INFO I-16

RESOLVED

Pause Docs Cite Wrong EVM Contract.

CATEGORY Documentation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/instructions/pause/set_paused.rs#L5 and related helpers

REVIEW Round 2 - Remediation Review

DESCRIPTION

The `console_oft` per-ID pause instruction comments describe EVM alignment as `PauseRBACUpgradeable.setPaused(...)` and `PauseRBACUpgradeable.setDefaultPaused(bool)`. Those functions are part of the per-ID pause implementation, `PauseByIDRBACUpgradeable`, while `PauseRBACUpgradeable` exposes only the global pause / unpause flow. The comments therefore point reviewers to the wrong EVM contract for the `IPauseByID` mapping. In addition, the README states that the `lz_receive` path is performing a pause check, which is not true.

RECOMMENDATION

Change the comments to reference `PauseByIDRBACUpgradeable.setPaused(...)` and `PauseByIDRBACUpgradeable.setDefaultPaused(bool)`. Also fix the readme.

RESOLUTION

Status: Resolved.

INFO I-17

RESOLVED

Misleading fee inverse edge cases.

CATEGORY Documentation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/instructions/fee_config/fee_config_view.rs#L69-L84

REVIEW Round 2 - Remediation Review

DESCRIPTION

`get_amount_before_fee()` is documented as the inverse of the fee calculation: given a post-fee amount, it returns the pre-fee amount needed. This description is incomplete for edge cases where the inverse is not unique or cannot be represented exactly. When `amount_after_fee == 0`, multiple pre-fee amounts can produce zero post-fee output depending on the configured fee and integer rounding. This is especially visible when `bps == MAX_FEE_BASIS_POINTS`, where every pre-fee amount produces zero post-fee output, so returning `0` is only one conservative representative value rather than a uniquely inferred pre-fee amount.

The helper also clamps results that exceed `u64::MAX` through `u64::try_from(amount_before_fee).unwrap_or(u64::MAX)`, which means the returned value may be a saturated cap rather than the exact mathematical pre-fee amount. These behaviors may be acceptable for quote-limit usage, but the current "inverse of fee" documentation does not describe them.

RECOMMENDATION

Update the documentation for `get_amount_before_fee()` to describe its edge-case behavior explicitly. State that `0` is returned as a conservative representative when the requested post-fee amount is zero or when a 100% fee makes every pre-fee amount produce zero post-fee output, and that values above `u64::MAX` are saturated to `u64::MAX`.

RESOLUTION

Status: Resolved.

INFO I-18

ACKNOWLEDGED

Arbitrary compose plans expose Executor tokens.

CATEGORY Validation LOCATION send_compose_if_needed() REVIEW Round 2 - Remediation Review

DESCRIPTION

OFT sender can cause the console_ofst receiver to create a compose job for an attacker controlled composer, and that composer V2 planner can request Executor signer placeholders in the final transaction On the compose side, the framework accepts arbitrary V2 planner output And each account can be a concrete pubkey, ALT entry, Executor payer, additional Executor signer, or context account There is no framework level rule that says: Payer may only be used in the known rent payer position, or Signer() may only be used for a specific predeclared account init pattern That means a malicious composer can return a plan that causes the Executor to place its own hot wallet / payer signer into attacker chosen instruction accounts Attack flow is 1) The attacker deploys a malicious Solana composer program with a composer account, for example, malicious_composer_store

2) The attacker sends OFT cross-chain with `send_to = malicious_composer_store` `compose_msg = attacker-controlled payload` 3) The honest console_ofst::lz_receive validates the LayerZero peer and clears the verified payload 4) Because the payload contains compose data, console_ofst calls Endpoint `send_compose` with to: malicious_composer_store 5) The Executor later discovers and executes the compose job. It asks the malicious composer for `lz_compose_types_v2` 6) The malicious composer returns a plan that includes the Executor payer or signer as a writable signer account. 7) The Executor signs and sends the real compose transaction 8) Inside `lz_compose`, the malicious composer uses the Executor payer signature to call `Token-2022 transfer_checked( from = executor token account, to = attacker token account, authority = executor payer )`

9) Because the Executor payer really signed the transaction, Token-2022 accepts the transfer This is not a malicious OApp only scenario. The console_ofst program is the one creating the compose job, and the attacker chooses the compose target through a normal user facing OFT message The next impact is mitigated: "the attacker can drain SOL up to the execution fee limit" The `preExecute/postExecute` checks cap the payer's balance loss, and the executor is already reimbursed for the amount spent through the fees paid upfront by the user, so this does not harm the executor The only impact is, Theft of non SOL assets if the Executor wallet also owns SPL token accounts The malicious `lz_compose` handler can then CPI into the SPL Token program and transfer tokens from the executor's token account we might need to downgrade this, If LayerZero executor wallets will always have SOL only

but we didn't find this enforced anywhere in the code / sdk

PoC `corepack enable corepack pnpm install --frozen-lockfile cd apps/project-types/console-ofst-app/contracts/solana corepack pnpm test:poc:native`

ACKNOWLEDGED

Arbitrary compose plans expose Executor tokens.

RECOMMENDATION

Fix in the Executor side after ALT resolution, before building the final compose transaction. Apply a guard in compose-types-v2.ts, inside `buildLzComposeExecutionPlan`, Create a helper function like `assertSafeComposePlan()`, and call it `const tables = await fetchAllAddressLookupTable({ rpc }, resp.alts) assertSafeComposePlan(executorProgram, tables, resp, payer)`

RESOLUTION

Status: Acknowledged.

LOW **L-01**

RESOLVED

Missing RateLimit PDA mismatch.

CATEGORY Logical Error

LOCATION rate_limit.rs

REVIEW Round 3 - Remediation Review

DESCRIPTION

The `apply_rate_limit()` function allows both `use_global_state == false` and missing per eid `RateLimit` PDA, when the following conditions are true. `// EVM parity: same early return gates as _applyRateLimit . if (!forward_enabled && (!backward_enabled || !net_accounting_enabled)) || (address_exemption_enabled && is_address_exempt) { return Ok(()); }` This can happen when: both directions are disabled current direction and net accounting are disabled the address is exempted In this case the send flow will be successfully executed.

The third condition, the address exemption, is not present

in `get_outbound_available_for_quote()` and `get_rate_limit_usages()` because there is no user provided. Even though the documentation claim A missing per-EID PDA is allowed here only when the real send/outflow path would also return before touching state. is technically not violated, this creates a mismatch between `quote_offt()/get_rate_limit_usages()` and `send()` where a user quoting would see failure, while he can successfully execute the send. The same issue is present on EVM, but there the quote will instead return stale value instead of failing.

RECOMMENDATION

Either document that the functions are unreliable in these specific cases or consider making the two paths behave the same way, for example don't include the exemption check when deciding whether to allow empty eid `RateLimit` PDA

RESOLUTION

Status: Resolved.

INFO I-01

RESOLVED

Informational Note Regarding Rate Limit Test.

CATEGORY Informational LOCATION console_ofst/src/state/rate_limit.rs#L760

REVIEW Round 3 - Remediation Review

DESCRIPTION

`test_missing_per_eid_state_quote_errors_when_net_accounting_needs_state` is intended to validate the quote-specific path for missing per-EID rate-limit state when outbound is disabled but inbound net accounting still requires state. However, the test calls `get_rate_limit_usages` instead of `get_outbound_available_for_quote` and does not cover intended quote functionality. Noting that this is only a test-suite issue and does not affect production behavior.

RECOMMENDATION

Update the test to call `get_outbound_available_for_quote` directly so the assertion covers the intended quote functionality.

RESOLUTION

Status: Resolved.

INFO I-02

RESOLVED

Rate-limit PDA requirement is underdocumented.

CATEGORY Documentation

LOCATION README.md

REVIEW Round 3 - Remediation Review

DESCRIPTION

The documentation states that default-and-override behavior is the same on both platforms, but this omits an important Solana account-model requirement for `RateLimit` state. On EVM, an uninitialized `rateLimits[id]` mapping entry behaves like a zeroed per-ID state while falling back to the default config. So if `useGlobalStateFlag == false` && `isGloballyDisabledFlag` and the rest of the preconditions about net accounting and rent exemption are not met, the code will use the empty state with the default config. On SVM, when `use_global_state` is `false` and the per-EID `RateLimit` PDA is missing, mutating paths such as `send()` and `lz_receive()` can return `RateLimitNotInitialized` if state must be updated.

RECOMMENDATION

Document this EVM/SVM difference

RESOLUTION

Status: Resolved.

INFO I-03

RESOLVED

Misleading Globally-Disabled Usage Doc.

CATEGORY Documentation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/state/rate_limit.rs#L237-L238 and related helpers

REVIEW Round 3 - Remediation Review

DESCRIPTION

The documentation on `get_rate_limit_usages` states that when rate limiting is globally disabled, "both availabilities are `u64::MAX`, but usage values still reflect decayed on-chain state" (rate_limit.rs#L237-L238). This is asserted unconditionally, but the function has a branch that contradicts it. When the resolved state is absent, `get_rate_limit_usages` takes the missing-state path (rate_limit.rs#L250-L257): `let Some(state) = state else { require!(oft_store.is_globally_disabled || (config.outbound_skips_state() && config.inbound_skips_state()), OFTError::RateLimitNotInitialized); return Ok(RateLimitUsages::unlimited()); }; RateLimitUsages::unlimited() hardcodes outbound_usage: 0 and inbound_usage: 0 (rate_limit.rs#L148-L155) and reads no on-chain state.`

As a result, for the same logical configuration (globally disabled), the reported usage differs depending on whether the optional per-EID rate-limit account exists: when it exists, usage is the decayed on-chain value the comment promises; when it is absent, usage is a hardcoded zero returned without touching state. The doc sentence describes only the first case. There is no behavioral defect: a missing per-EID account has never recorded any in-flight amount, so its correctly-decayed usage genuinely is zero, and returning zero is consistent with the EVM mapping model where an uninitialized `id` reads as a zero-usage entry. The impact is confined to documentation precision. An integrator or reviewer relying on the comment could assume the globally-disabled usages view always performs a decay computation over stored state, when the missing-account branch instead returns fixed zeros.

RECOMMENDATION

Update the docs to reflect the actual behavior, qualifying the sentence so it applies only when state exists.

RESOLUTION

Status: Resolved.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

About Guardian.

Founded in 2022 by DeFi experts, Guardian is a leading audit firm in the DeFi smart contract space. With every audit report, Guardian upholds best-in-class security while achieving our mission to relentlessly secure DeFi.